

Лабораторна робота №1

Реалізація та перевірка зворотного поширення похибки

Герасименко Денис, група КМ-33

4 жовтня 2026 р.

1 Мета роботи

Реалізувати зворотне поширення похибки вручну та перевірити отримані градієнти двома способами: за допомогою PyTorch autograd і чисельного диференціювання. Після виконання роботи ви маєте вміти пояснити, звідки береться кожен градієнт, які проміжні значення потрібні для його обчислення та як виявити помилку у зворотному проході. Основні перевірки виконуються до навчання; мінімальна точність класифікації не встановлюється.

2 Формули зворотного проходу та розмірності

Для навчальної вибірки з $N = 105$ об'єктів розмірності матриць такі: Вхідні дані $X \in \mathbb{R}^{105 \times 4}$, ваги першого шару $W_1 \in \mathbb{R}^{4 \times 8}$, $b_1 \in \mathbb{R}^8$. Ваги другого шару $W_2 \in \mathbb{R}^{8 \times 3}$, $b_2 \in \mathbb{R}^3$. Мітки класів у one-hot представленні $Y \in \mathbb{R}^{105 \times 3}$.

У програмній реалізації мітки класів зберігаються у вигляді цілочисельних індексів $y \in \{0, 1, 2\}^{105}$, тому матриця Y явно не створюється. Еквівалентне віднімання одиниці для правильного класу виконується операцією `dZ2[np.arange(N), y] -= 1.0`.

Усі обчислення виконуються у форматі `float64` відповідно до вимог лабораторної роботи.

Прямий прохід

$$\begin{aligned} Z_1 &= XW_1 + b_1 & (105 \times 8) \\ A_1 &= \max(0, Z_1) & (105 \times 8) \\ Z_2 &= A_1W_2 + b_2 & (105 \times 3) \\ P &= \text{softmax}(Z_2) & (105 \times 3) \end{aligned}$$

Зворотний прохід

Градiєнт крос-ентропії за логітами Z_2 :

$$dZ_2 = \frac{1}{N}(P - Y) \quad (105 \times 3) \quad (1)$$

Градiєнти параметрів вихідного шару:

$$dW_2 = A_1^T dZ_2 \quad (8 \times 3) \quad (2)$$

$$db_2 = \sum_{i=1}^N (dZ_2)_i \quad (3,) \quad (3)$$

Поширення похибки через прихований шар та активацію ReLU:

$$dA_1 = dZ_2 W_2^T \quad (105 \times 8) \quad (4)$$

$$dZ_1 = dA_1 \odot (Z_1 > 0) \quad (105 \times 8) \quad (5)$$

Градiєнти параметрів вхідного шару:

$$dW_1 = X^T dZ_1 \quad (4 \times 8) \quad (6)$$

$$db_1 = \sum_{i=1}^N (dZ_1)_i \quad (8,) \quad (7)$$

Пояснення збережених проміжних значень

Для коректного обчислення градієнтів під час зворотного проходу необхідно зберегти:

- X : використовується для обчислення dW_1 .
- Z_1 : необхідне для обчислення похідної функції активації ReLU (індикаторна функція $Z_1 > 0$).
- A_1 : використовується для обчислення градієнта ваг вихідного шару dW_2 .
- P (probs) та y : необхідні для обчислення первинного градієнта dZ_2 .

3 Зв'язка з PyTorch

Втрата NumPy: 1.456200780114

Втрата PyTorch: 1.456200780114

	Макс. абсолютна різниця	Result
Loss	2.2204×10^{-16}	True
Градiєнт W_1	2.7756×10^{-17}	True
Градiєнт b_1	3.2960×10^{-17}	True
Градiєнт W_2	1.1102×10^{-16}	True
Градiєнт b_2	1.2490×10^{-16}	True

Усі різниці не перевищують допустимий поріг 10^{-12} .

4 Чисельна перевірка градієнтів

Усі різниці відповідають критерію $\leq 10^{-7}$.

Параметр	backward()	Похідна	Абс. різниця	Result
$W_1[0, 0]$	-0.01992018	-0.01992018	6.0343×10^{-11}	True
$b_1[0]$	-0.07056611	-0.07056611	3.6327×10^{-11}	True
$W_2[0, 0]$	0.13523021	0.13523021	9.2180×10^{-11}	True
$b_2[0]$	0.08552164	0.08552164	1.4603×10^{-10}	True

5 Дослід із навмисною помилкою

Опис модифікації: У функції зворотного проходу було закоментовано ділення на кількість об'єктів у батчі ($N = 105$) під час обчислення градієнта за логітами ($dZ2 \neq N$).

Прогноз:

1. Значення функції втрат не зміниться, оскільки воно обчислюється на прямому проході, який залишився без змін.
2. Усі ненульові градієнти стануть рівно в 105 разів більшими порівняно з еталонними значеннями.
3. Обидві перевірки повинні виявити помилку: PyTorch використовує ту саму усереднену функцію втрат, а чисельна похідна безпосередньо наближає похідну саме цієї функції втрат.

Результати дослідів:

- Функція втрат залишилася математично незмінною (1.456200780114).
- Відношення модифікованого градієнта до правильного склало рівно 105.0.
- Перевірка PyTorch виявила макс. різницю 1.85×10^1 (Пройдено: False).
- Чисельна перевірка виявила різницю 2.07 (Пройдено: False).

Прогноз повністю справдився. Звірка з незалежною реалізацією в PyTorch та чисельною похідною дозволила виявити помилку масштабування градієнтів.

6 Висновки

Під час виконання роботи було успішно реалізовано алгоритм зворотного поширення похибки з нуля без використання механізмів автоматичного диференціювання в NumPy-реалізації. Успішне проходження перевірки з PyTorch та чисельного диференціювання підтвердило коректність реалізації обчислення градієнтів. Дослід із навмисною помилкою підтвердив важливість незалежної перевірки реалізації градієнтів перед етапом навчання.